

AI SKIN INTELLIGENCE

Personalized Skincare Planner

Documentation, Methodology, System Architecture & User Guide

Live Application: <https://ai-skin-intelligence-personalized-skincare-planner-5edjzgn7u.vercel.app/>

Backend API: <https://ai-skin-intelligence-personalized-puf3.onrender.com/>

1. Project Overview

AI Skin Intelligence is a web-based personalized skincare platform designed to help users understand their skincare needs and receive structured, personalized recommendations. The platform combines a modern web interface, secure authentication, backend APIs, database management, skincare assessment, recommendation logic, progress tracking, notifications and dashboard features in one system.

2. Objectives

- Provide a simple and user-friendly skincare assessment experience.
- Create personalized skincare recommendations based on user-provided information.
- Support secure account registration and login using email/password and Google OAuth.
- Maintain user profiles, assessments, routines, progress and related records.
- Provide dashboards and reports that help users monitor their skincare journey.
- Provide role-based functionality for users, consultants, dermatologists and administrators.

3. Key Technologies

Layer	Technology / Tool	Purpose
Frontend	React.js, Vite, JavaScript, CSS	Interactive user interface and client-side routing
Backend	Python, FastAPI, Uvicorn	REST APIs, authentication and business logic
Database	PostgreSQL, SQLAlchemy	Persistent storage of users and application data
Authentication	JWT + Google OAuth	Secure local login and Google-based authentication
Deployment	Vercel + Render	Frontend and backend cloud deployment
ML / AI	Python / TensorFlow components where applicable	Data-driven skincare intelligence features

4. Methodology

The project follows a modular, API-driven methodology. The frontend collects user input and communicates with the FastAPI backend through HTTP requests. The backend validates requests, performs authentication and business

processing, and communicates with PostgreSQL through SQLAlchemy. The resulting data is returned to the frontend and displayed through dashboards, recommendations and user-specific pages.

4.1 Development Workflow

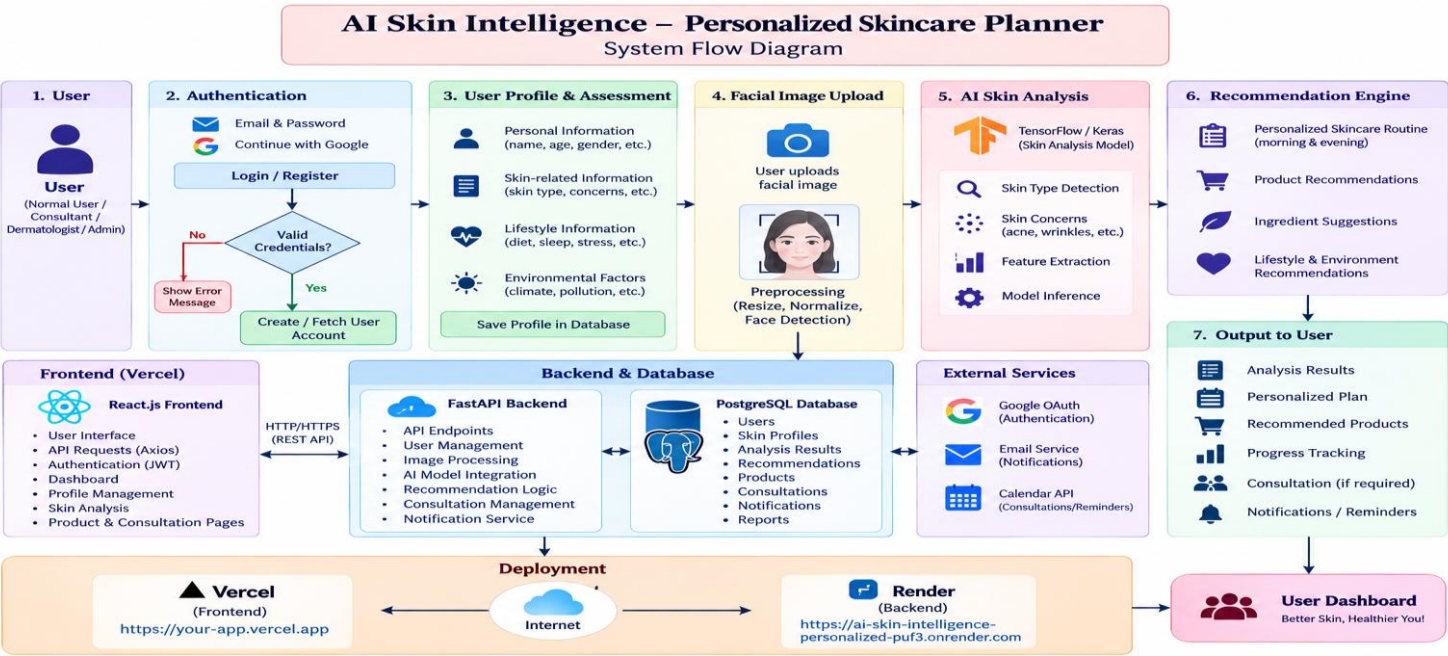
- Requirement Analysis – Identify user roles, skincare assessment requirements, authentication needs and dashboard functions.
- UI/UX Design – Design pages for landing, registration, login, assessment, dashboard, recommendations and account management.
- Frontend Development – Implement React components, routes, forms, protected pages and API integration.
- Backend Development – Implement FastAPI routers, schemas, models, authentication and application services.
- Database Integration – Define SQLAlchemy models and connect the application to PostgreSQL.
- Authentication – Implement password hashing, JWT token generation and Google OAuth flow.
- Testing – Test registration, login, protected routes, API responses, database operations and deployment behavior.
- Deployment – Deploy the frontend on Vercel and backend API on Render, then configure environment variables.

5. System Architecture

The system uses a three-tier architecture with a separate frontend, backend API and database. This separation improves maintainability and allows the frontend and backend to be deployed independently.

Component	Responsibility
React Frontend	User interface, navigation, forms, dashboards and displaying API results.
FastAPI Backend	Authentication, validation, business logic, OAuth callback handling and REST endpoints.
PostgreSQL	Stores users, roles, profiles, assessments, routines, notifications and application records.
Google OAuth	Provides Google-based authentication and user identity information.
Vercel	Hosts the production frontend.
Render	Hosts the production backend API.

5.1 Architecture Flow



6. Authentication Methodology

For local authentication, the user registers with a name, email and password. The password is hashed before storage. During login, the submitted password is verified against the stored hash, and a JWT access token is generated after successful authentication. Google OAuth follows an authorization-code flow and creates or retrieves the corresponding local user account before issuing the application's JWT.

8. Environment Configuration

Production deployment requires frontend and backend environment variables to point to the correct services. Typical variables include VITE_API_URL on the frontend and DATABASE_URL, FRONTEND_URL, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET and GOOGLE_REDIRECT_URI on the backend. Local development uses localhost URLs, while production uses the deployed Vercel and Render URLs.

9. Performance Matrix

The following matrix defines the main performance and quality indicators for evaluating the deployed system. Values should be updated with measured results from browser DevTools, API testing tools or application monitoring; the table intentionally avoids presenting unverified measurements as actual results.

Metric	What to Measure	Target / Interpretation	Measurement Method
Frontend Load Time	Time until the main application is usable	Keep initial load low and optimize large assets	Chrome DevTools / Lighthouse
API Response Time	Time taken by backend endpoints	Fast and consistent responses under normal load	Browser Network tab / Postman
Login Success Rate	Successful valid login requests	100% for valid configured credentials	Functional test cases
Registration Success	Successful account creation	Valid users should be created without duplicate accounts	API/UI testing
Google OAuth	Successful OAuth login and callback	Successful redirect, callback and token creation	End-to-end browser test
Database Response	Query and transaction time	Consistent performance without unnecessary queries	Backend logs / DB monitoring
Error Rate	Failed API/application requests	Minimize unexpected 4xx/5xx errors	Browser console / backend logs
Availability	Whether deployed services are reachable	Stable access during normal operation	Deployment monitoring
Usability	Ease of completing core tasks	Users can register, login and navigate without confusion	User testing / checklist
Security	Password and token handling	Passwords hashed; secrets kept in environment variables	Code review / configuration review

10. Testing Checklist

- Register a new user with valid details.
- Attempt registration using an already registered email.

- Login with valid credentials.
- Verify invalid credentials are rejected.
- Verify protected pages cannot be opened without authentication.
- Test Google OAuth using the configured production redirect URI.
- Verify user information is correctly retrieved from the database.
- Verify role-based pages and APIs reject unauthorized roles.
- Test profile update and password-change functionality.
- Check frontend-to-backend communication in both local and production environments.

11. Deployment Summary

The production frontend is deployed through Vercel and communicates with the FastAPI backend deployed through Render. Environment variables are used to separate local-development configuration from production configuration. For Google OAuth, the redirect URI registered in Google Cloud must exactly match the backend callback URL used by the deployed environment.

12. Conclusion

AI Skin Intelligence provides an integrated platform for personalized skincare planning using a modular frontend-backend-database architecture. The methodology supports secure authentication, structured data management and personalized application workflows. The documentation and performance matrix provide a practical reference for users, developers and evaluators to understand, operate and assess the system.